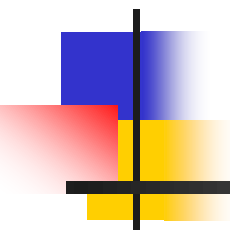


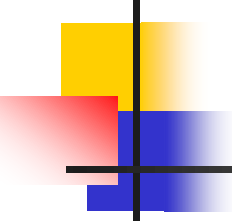
08. File I/O in Java



File I/O APIs

Reader and Writer APIs

File Management



Questions: Exception Handling

- Is the following code legal?

```
try {
```

```
} finally {
```

```
}
```

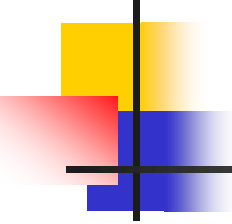
- Is there anything wrong with this exception handler as written?
Will this code compile?

```
try {
```

```
} catch (Exception e) {
```

```
} catch (ArithmeticException a) {
```

```
}
```



JAVA STREAMS

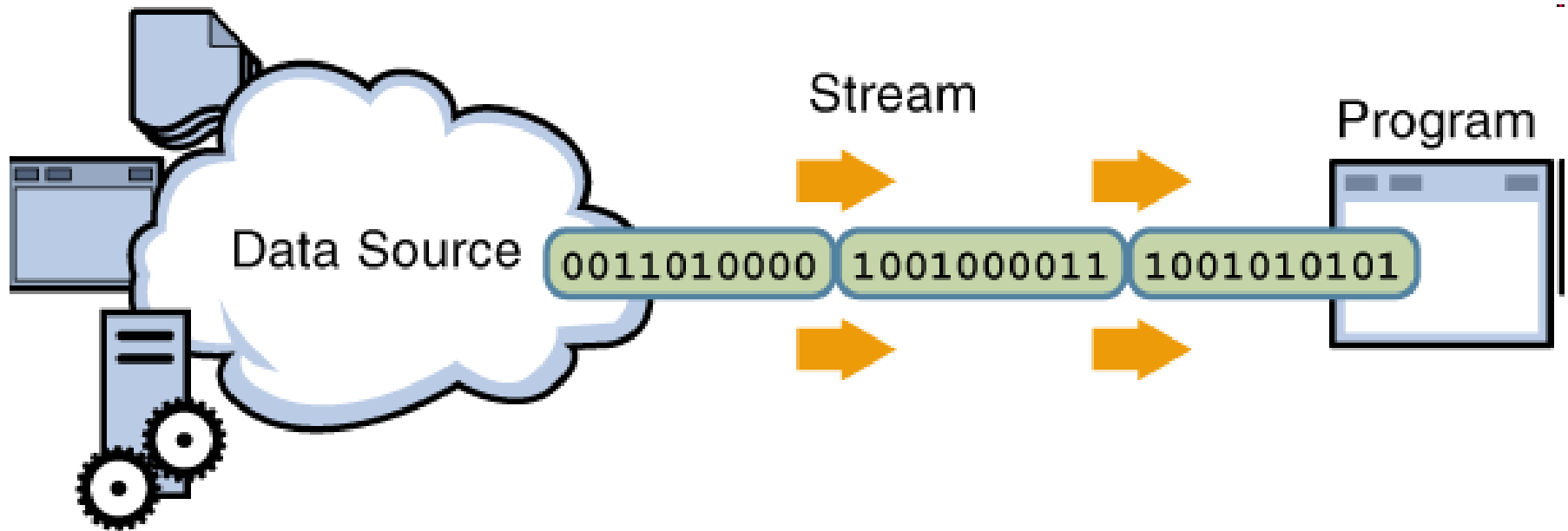
- A stream is the path traveled by data in a program.
- Depending upon the direction of the data, streams are classified into TWO categories:
 - Input Streams
 - Output Streams

Input Streams:

- **Bringing Information In** To bring in information, open a stream on an information source(a file, memory, or socket) and read the information serially.



Input Streams:

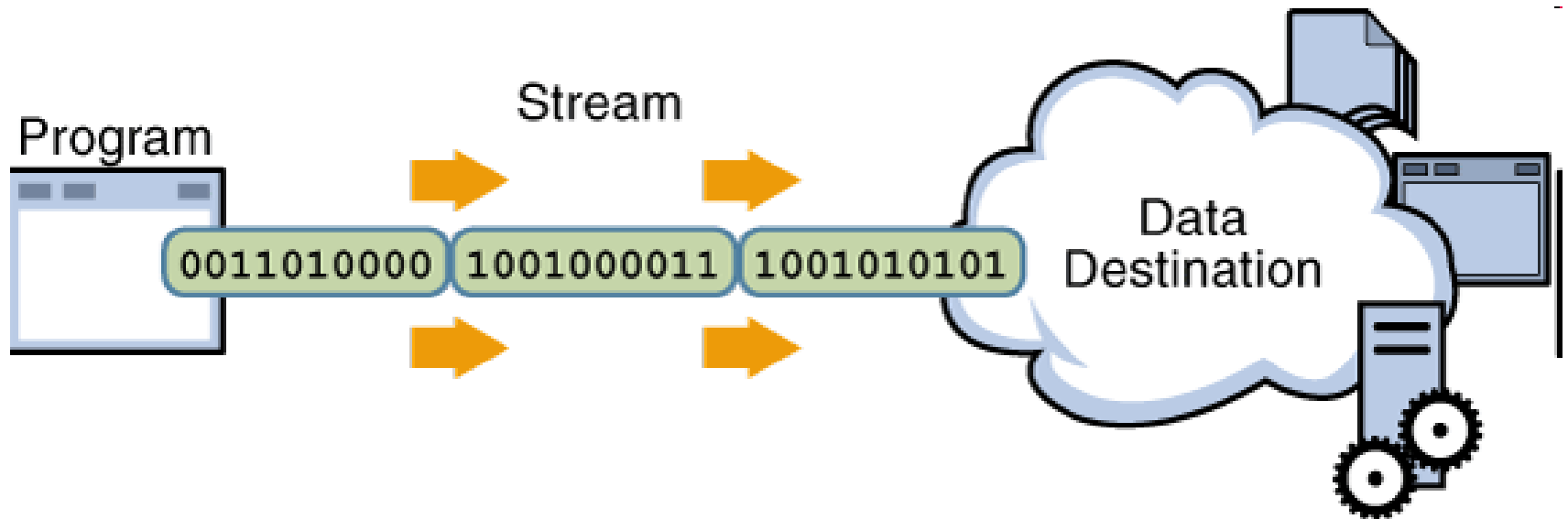


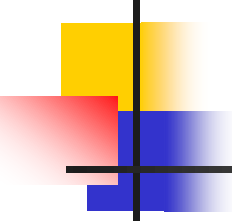
Output Streams:

- **Sending Information Out** A program can send information to an external destination by opening a stream to a destination and writing the information out serially.



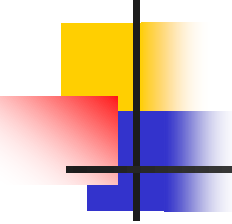
Output Streams:





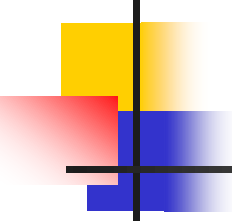
Using a Stream

- Import the package *java.io*, which contains a collection of stream classes.
- Create an object of input/output stream that is associated with the data source / destination.



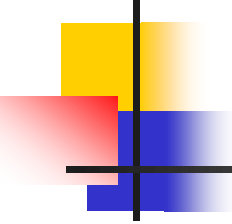
Using a Stream

- Read/write data using this object's *read()*/*write()* methods.
- Finally close the stream by calling its *close()* method.



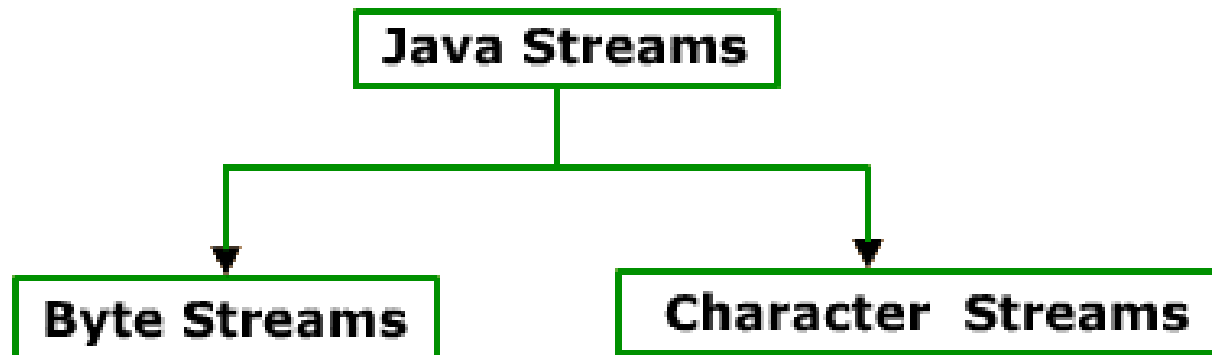
Predefined standard streams

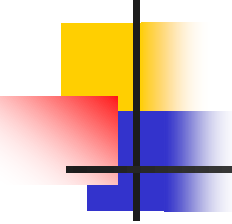
- ***System.in*** - Keyboard input
 - The "standard" input stream. This stream is already open and ready to supply input data.
- ***System.out*** - Display output
 - The "standard" output stream. This stream is already open and ready to accept output data.
- ***System.err*** - Error stream
 - The "standard" error stream. This stream is already open and ready to accept output data.



Stream classification

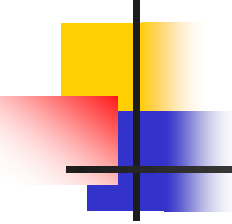
- Depending upon data type on which they operate, Java streams are classified into 2 categories:



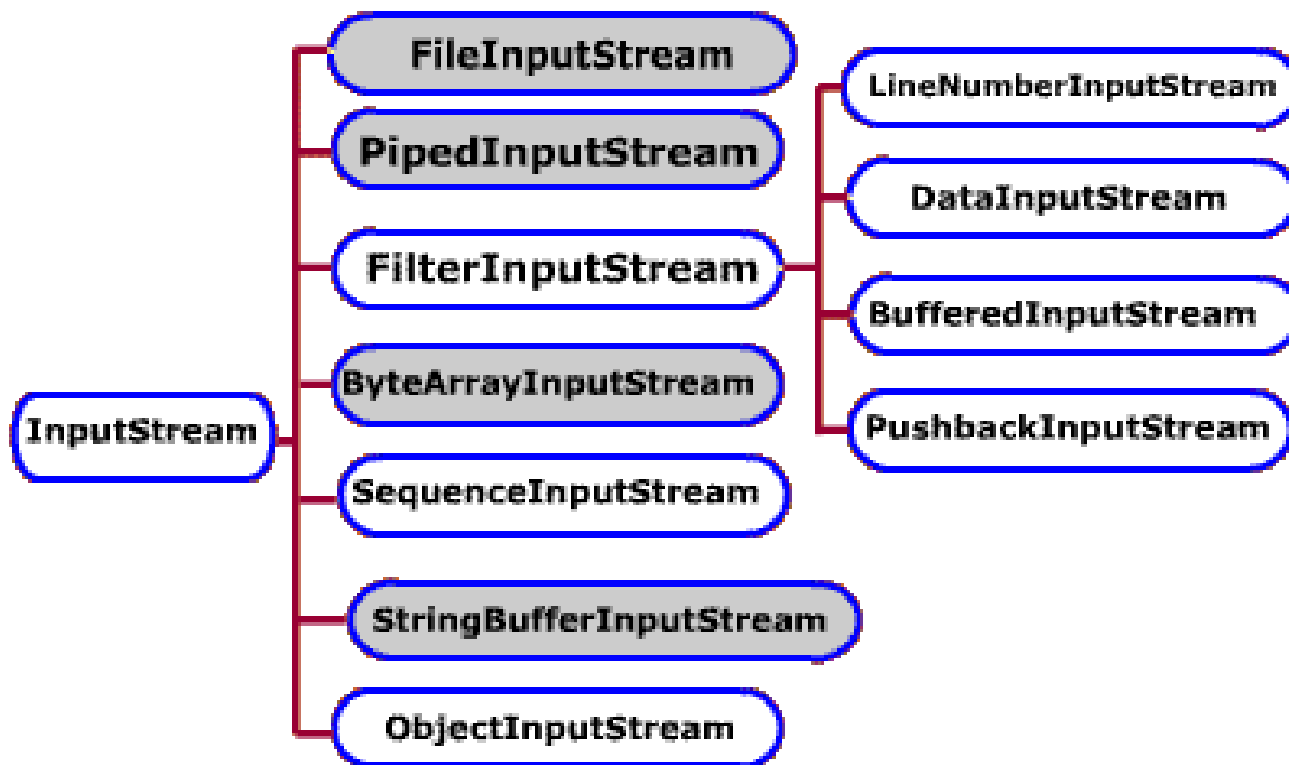


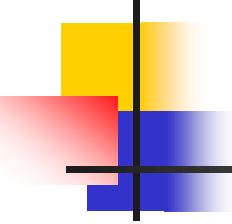
Byte Streams

- Programs use *byte streams* to perform input and output of 8-bit bytes. All byte stream classes are descended from InputStream and OutputStream.

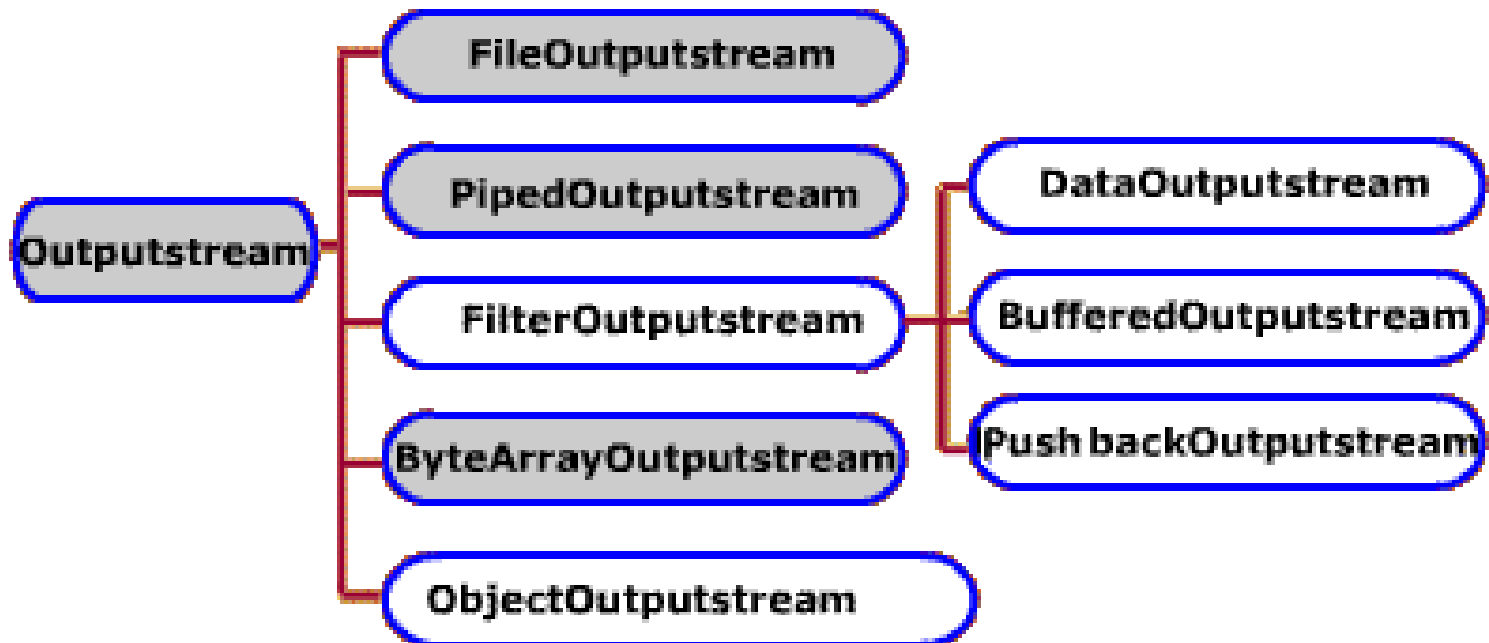


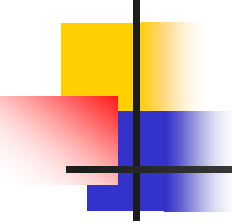
Byte Stream Class Hierarchy





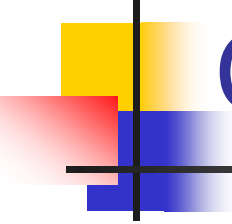
Byte Stream Class Hierarchy



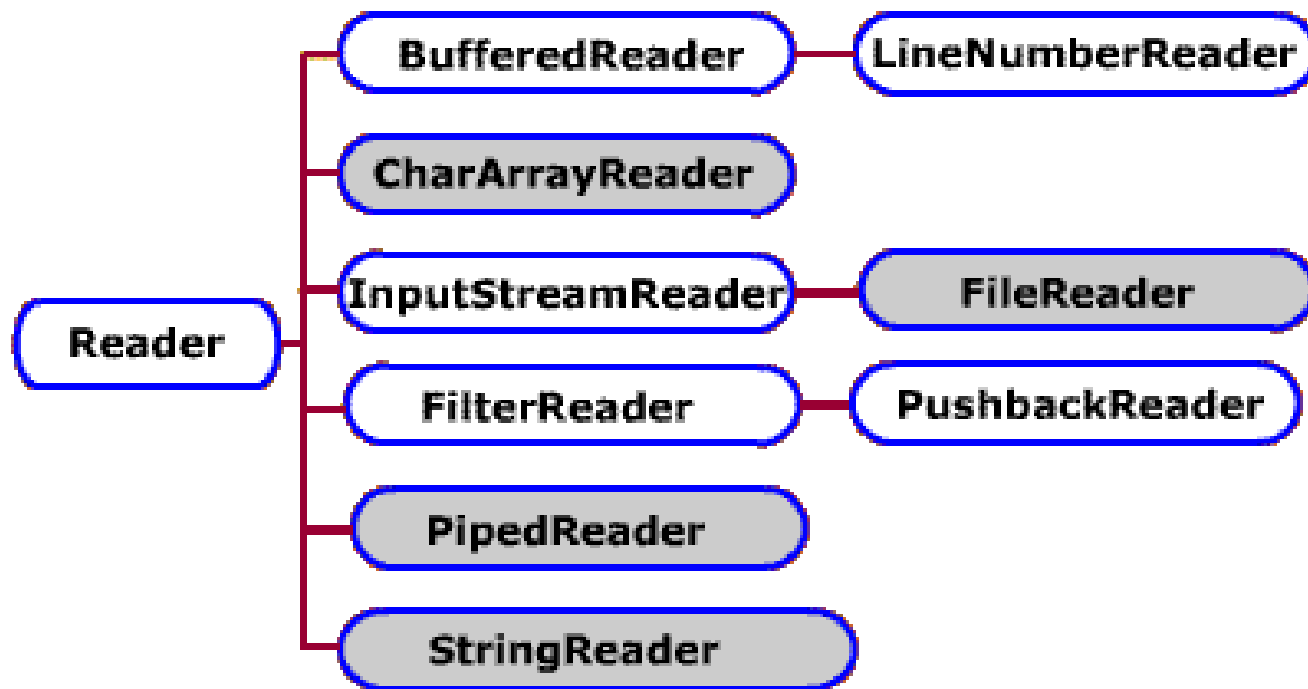


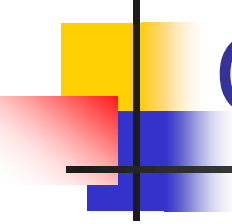
Character Streams

- The Java platform stores character values using Unicode conventions. Character stream I/O automatically translates this internal format to and from the local character set. All character stream classes are descended from Reader and Writer

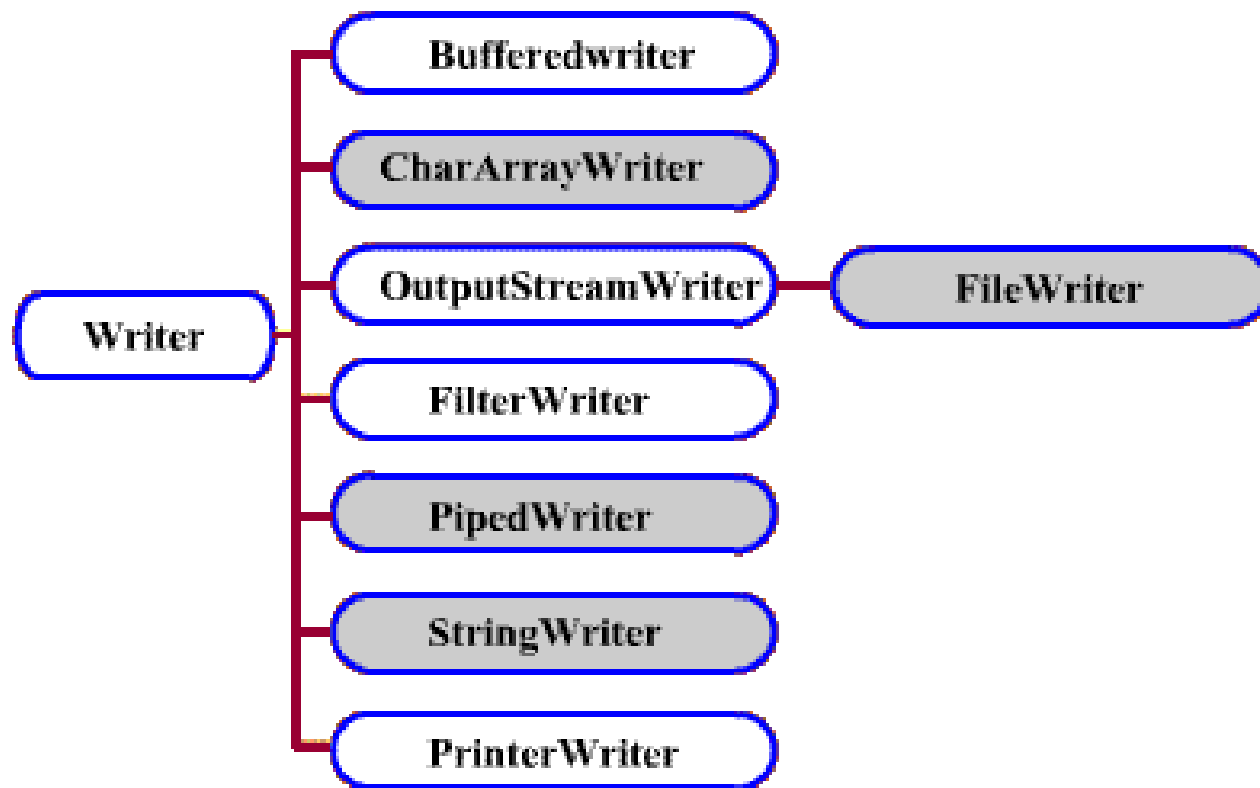


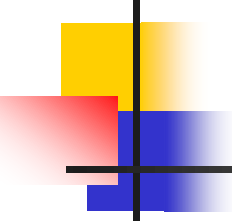
Character Stream Class Hierarchy





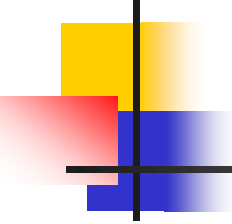
Character Stream Class Hierarchy





Key Differences

Criteria	<code>InputStream</code> (Byte Stream)	<code>Reader</code> (Character Stream)
Type of Data	Raw binary data (e.g., images, audio, video, PDFs, etc.)	Textual data (e.g., <code>.txt</code> , <code>.csv</code> , HTML, XML, etc.)
Unit of Reading	Bytes (8-bit)	Characters (16-bit, Unicode)
Use Case	Non-text, binary file handling	Text file handling
Character Encoding	No concern for character encoding (reads raw bytes as they are)	Handles character encoding (UTF-8, UTF-16, etc.) to interpret characters
Example Classes	<code>FileInputStream</code> , <code>BufferedInputStream</code> , <code>ByteArrayInputStream</code>	<code>FileReader</code> , <code>BufferedReader</code> , <code>CharArrayReader</code>
Corruption Risk	None for binary files	Risk of data corruption if used for binary data (interprets bytes as characters)



File Streams

- FileInputStream ? read()
- FileOutputStream? write()
- FileReader
- FileWriter

■ File Streams

```
try {  
    FileInputStream fis = new FileInputStream("pic.gif");  
    FileOutputStream fos= new FileOutputStream("new.gif");  
    // FOS fos = new FOS("new.gif",true); -- for appending.  
    boolean eof = false;  
  
    while(!eof) {  
        int input = fis.read();  
        fos.write(input);  
        if(input == -1) eof = true;  
    }  
  
    fis.close(); fos.close();  
} catch(IOException e) { }
```

■ Buffering File Streams

```
FileInputStream fis = new FileInputStream("pic.gif");  
BufferedInputStream bis = new BufferedInputStream(fis);  
                        // new BufferedInputStream(fis, bufSize);
```

Similarly,

```
FileOutputStream fos = new FileOutputStream("pic.gif");  
BufferedOutputStream bos = new BufferedOutputStream(fos);  
fos.close();  
bos.close();
```

■ **FileReaders and FileWriters**

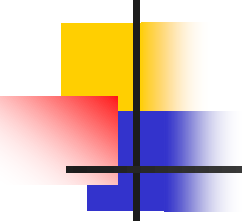
```
FileReader file = new FileReader("Test.java");  
BufferedReader buf = new BufferedReader(file);
```

Use `readLine()` method of `buf` to read one line at a time.

i.e., `String line = buf.readLine();`

Similarly,

```
FileWriter file = new FileWriter("Test1.java");  
BufferedWriterr buf = new BufferedWriterr(file);  
buf.write(line);
```



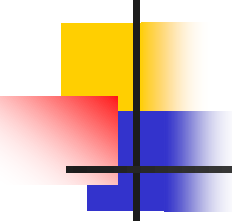
File class

- We can create an instance of a class '***File***' from a file name string:

```
File fileObj = new File (" Person.java ");
```

- We can get more information about the file using this file reference.
- We can also use this reference to create a stream object.

```
FileInputStream fis = new FileInputStream(fileObj);
```



Important methods of *File* class:

String getAbsolutePath()
String getParent()
String[] list()
long length()
URL toURL()

All Return Boolean upon invocation

- canRead()
- canWrite()
- delete()
- exists()
- mkdir()
- renameTo()